

TÀI LIỆU THIẾT KẾ HỆ THỐNG

Hybrid Document-Graph Store: Medical Knowledge Base | Trần Hữu Trung – N23DCCN200 – D23CQC�N03-N
Giảng viên: Lê Hà Thanh

Section 1: Kiến Trúc Hệ Thống (4 Tầng)

Tầng	Tên tầng	Thành phần chính	Chức năng
4	User Interface Layer	Flask Templates, D3.js	Hiện thị kết quả, visualize đồ thị
3	API Layer	Flask REST Endpoints	Routing: /api/query, /api/graph/traverse, /api/analyze/join-cost
2	Engine Layer	Document Engine + Graph Engine + Hybrid Engine	Xử lý BM25F, METIS traversal, combined scoring, join cost
1	Data Layer	JSON files, Whoosh Index, partitions.txt	Lưu trữ và truy xuất dữ liệu

Luồng xử lý: POST /api/query → Hybrid Engine song song hóa BM25F search và Graph traversal → merge theo $\text{combined_score} = w_1 \times \text{norm}(\text{BM25F}) + w_2 \times \text{graph_importance}$ → trả JSON.

Section 2: Thiết Kế Document Engine (Whoosh BM25F)

Schema BM25F – 5 trường searchable với analyzer StemmingAnalyzer, trọng số bằng nhau:

Trường	Boost	Kiểu	Ghi chú
chief_complaint	1.0	TEXT	Triệu chứng chính
symptoms	1.0	TEXT	Danh sách triệu chứng
initial_diagnosis	1.0	TEXT	Chẩn đoán bác sĩ
medical_history	1.0	TEXT	Tiền sử bệnh án
full_text	1.0	TEXT	Toàn văn (tổng hợp chief_complaint, symptoms, medical_history, initial_diagnosis)
department	–	KEYWORD	Filter theo khoa
severity	–	NUMERIC	Filter mức độ nghiêm trọng (1-5)

Công thức $\text{score}(q,d) = [\text{BM25F}]$ với $k1=1.2$, $b=0.75$, $\text{boost}_f=1.0$ cho tất cả các trường searchable (equal weighting).

Section 3: Thiết Kế Graph Engine (METIS Edge-Cut, 4 Phân Vùng)

Cấu trúc đồ thị: 200 nút, ~800 cạnh có trọng số

Loại nút	Số lượng	Loại cạnh	Trọng số cạnh
Disease	~51	disease_to_disease / disease_to_symptom / disease_to_field	0.0–1.0 (correlation strength)
Symptom	~70	disease_to_symptom / symptom_to_symptom	0.0–1.0
Medical Field	10	disease_to_field	1.0 (hierarchical)
Medication	~40	disease_to_drug	0.0–1.0

METIS Recursive Bisection – 3 bước:

- Bước 1 – Coarsening:** Heavy Edge Matching giảm đồ thị $G \rightarrow G'$ cho đến $|G'| < 100$ nút.
- Bước 2 – Initial Partitioning:** Greedy Graph Growing chia G' thành $k=4$ phân vùng ban đầu.
- Bước 3 – Uncoarsening & Refinement:** Mở rộng ngược, tinh chỉnh Kernighan-Lin tối thiểu hóa edge-cut.

Chỉ số mục tiêu: $\text{ECR} = |E_{\text{cut}}|/|E_{\text{total}}| < 0.25$ | Balance Factor = $\max(|P_i|)/\text{avg}(|P_i|) \leq 1.2$ | Intra-Partition Density > 0.3

Section 4: Thiết Kế Hybrid Engine (Combined Scoring + 3 Chiến Lược Join)

Công thức điểm kết hợp:

$\text{combined_score} = \text{text_weight} \times \text{normalize}(\text{BM25F_score}) + \text{graph_weight} \times \text{graph_importance}$

graph_importance: 0-hop (cùng field) = 1.0 | 1-hop = 0.8 | 2-hop = 0.5 | 3-hop = 0.2 | ngoài đồ thị = 0.1

So sánh 3 chiến lược Join:

Chiến lược	Phức tạp	Tổng chi phí	Dùng khi
Filter-After-Join	$O(N \cdot G)$	IO cao, CPU thấp	N nhỏ (<50 docs), query mơ hồ
Index Nested-Loop	$O(k \cdot \log G)$	IO trung bình	k vừa (20–50), graph lớn
Hash Join (tối ưu)	$O(G + k)$	CPU cao, IO thấp	k > 50, graph và doc đều lớn

Mô hình chi phí: $Total_Cost = IO_Cost + CPU_Cost + Comm_Cost$ với $Comm_Cost = |E_cut| \times network_latency$ (1ms/hop local).

Section 5: Các Quyết Định Thiết Kế Quan Trọng

Quyết định	Lựa chọn	Lý do	Phương án bị loại
Partitioning	Edge-Cut (METIS)	Đồ thị y tế thưa (density~0.04); ECR<0.25 đạt được; METIS hỗ trợ trực tiếp	Vertex-Cut – phù hợp đồ thị dày (social graph)
Ranking	BM25F	Hỗ trợ multi-field search (5 fields); saturation effect; length normalization tốt hơn TF-IDF	TF-IDF – không có field-weight native; BM25F phù hợp hơn văn bản y tế
Graph traversal	BFS (mặc định)	Tính hop count chính xác nhất; đồ thị thưa nên BFS nhanh; $O(V+E)$	Dijkstra chỉ dùng khi cần weighted shortest path
Join key	Disease name (string)	Semantic key xuất hiện ở cả initial_diagnosis field và Disease node name	ID-based join – không có shared ID schema